Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ **Информатика и системы управления**

КАФЕДРА **Компьютерные системы и сети**

НАПРАВЛЕНИЕ ПОДГОТОВКИ **09.03.01/03 Вычислительные машины, комплексы, системы и сети**

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ БАКАЛАВРА НА ТЕМУ:

Компилятор стекового языка программирования

Студент ИУ6-83Б (Группа) _____ (Подпись, дата) В.К. Залыгин (И.О. Фамилия)

Руководитель ВКРБ _____ (Подпись, дата) Б.И. Бычков (И.О. Фамилия)

Нормоконтролер _____ (Подпись, дата) _____ (И.О. Фамилия)

2026 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ

Заведующий кафедрой ___ИУ6___

_____ А.В. Пролетарский

« __ » _____ 2026 г.

З А Д А Н И Е
на выполнение выпускной квалификационной работы бакалавра

Студент группы ИУ6-83Б

Залыгин Вячеслав Константинович
(Фамилия, имя, отчество)

Тема квалификационной работы: Компилятор стекового языка программирования

При выполнении ВКР:

Используются / Не используются	Да/Нет
1) Литературные источники и документы, имеющие гриф секретности	Нет
2) Литературные источники и документы, имеющие пометку «Для служебного пользования», иных пометок, запрещающих открытое опубликование	Нет
3) Служебные материалы других организаций	Нет
4) Результаты НИР (ОКР), выполняемой в МГТУ им. Н.Э. Баумана	Нет
5) Материалы по незавершенным исследованиям или материалы по завершенным исследованиям, но ещё не опубликованные в открытой печати	Нет

Тема квалификационной работы утверждена распоряжением по факультету:	
Название факультета:	Информатика и системы управления
Дата и рег. номер распоряжения:	№ 30.30-30/1781 от 28.11.2025 г.

Часть 1. Исследовательская

Провести анализ синтаксиса и семантики различных стековых языков программирования, выделить и сравнить их особенности. Выполнить анализ средств статического анализа стековых языков программирования. Провести анализ устройства байткода и модели исполнения WebAssembly.

Часть 2. Конструкторская

Осуществить выбор технологии и средств разработки для программного обеспечения. Разработать структуру программного обеспечения и определить его компоненты. Выполнить проектирование программных компонентов. Реализовать компоненты программного обеспечение с использованием выбранных средств разработки.

Выполнить сборку программного обеспечения и произвести его комплексное тестирование.

Часть 3. Технологическая

Разработать технологию построения SSA-представления текстов программ на стековых языках.

Оформление квалификационной работы:

Расчетно-пояснительная записка на 55–65 листах формата А4.

Перечень графического (иллюстративного) материала (чертежи, плакаты, слайды и т.п.)

1. Схема процесса трансляции программы в байткод WebAssembly и ее исполнения.
2. Результаты анализа стековых языков программирования.
3. Схема алгоритма вывода типов.
4. Функциональная диаграмма программного обеспечения.
5. Диаграмма компоновки.
6. Схема алгоритма построения SSA-представления текстов программ на стековых языках.

Дата выдачи задания « 1 » сентября 2025 г.

В соответствии с учебным планом выпускную квалификационную работу выполнить в полном объеме в срок до « 1 » июня 2026 г.

Руководитель квалификационной работы	_____	Б.И. Бычков
	(Подпись, дата)	(И.О. Фамилия)
Студент	_____	В.К. Залыгин
	(Подпись, дата)	(И.О. Фамилия)

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика и системы управления УТВЕРЖДАЮ

КАФЕДРА Компьютерные системы и сети Заведующий кафедрой ИУ6

ГРУППА ИУ6-83Б _____ А.В. Пролетарский

« _____ » _____ 2026 г.

КАЛЕНДАРНЫЙ ПЛАН
выполнения выпускной квалификационной работы бакалавра

студента: Залыгина Вячеслава Константиновича
(фамилия, имя, отчество)

Тема квалификационной работы: Компилятор стекового языка программирования

№ п/п	Наименование этапов выпускной квалификационной работы	Сроки выполнения этапов		Отметка о выполнении	
		план	факт	Должность	ФИО, подпись
1.	Задание на выполнение работы. Формулирование проблемы, цели и задач работы	09.2025 Планируемая дата		Руководитель ВКР	Б.И. Бычков
2.	1 часть Исследовательская	12.2025 Планируемая дата		Руководитель ВКР	Б.И. Бычков
3.	Утверждение окончательных формулировок решаемой проблемы, цели работы и перечня задач	02.2026 Планируемая дата		Заведующий кафедрой	А.В. Пролетарский
4.	2 часть Конструкторская	05.2026 Планируемая дата		Руководитель ВКР	Б.И. Бычков
5.	3 часть Технологическая	05.2026 Планируемая дата		Руководитель ВКР	Б.И. Бычков
6.	1-я редакция работы	05.2026 Планируемая дата		Руководитель ВКР	Б.И. Бычков
7.	Подготовка доклада и презентации	06.2026 Планируемая дата		Руководитель ВКР	Б.И. Бычков
8.	Отзыв руководителя	06.2026 Планируемая дата		Руководитель ВКР	Б.И. Бычков
9.	Нормоконтроль	06.2026 Планируемая дата		Нормоконтролер	
10.	Внешняя рецензия	06.2026 Планируемая дата		Руководитель ВКР	Б.И. Бычков
11.	Защита работы на ГЭК	06.2026 Планируемая дата		Руководитель ВКР	Б.И. Бычков

Руководитель квалификационной работы _____ Б.И. Бычков
(подпись, дата) (И.О. Фамилия)

Студент _____ В.К. Залыгин
(подпись, дата) (И.О. Фамилия)

1 Аннотация

Расчетно-пояснительная записка посвящена процессу разработки компилятора стекового языка программирования. В исследовательской части рассмотрены различные особенности стековых языков программирования, проведено их сравнение, сделаны выводы об области применимости стековых языков. Затем описаны алгоритмы типизации для стековых языков, а также рассмотрена структура и модель исполнения виртуальной машины WebAssembly. В конструкторской части определена структура компилятора, выполнены проектирование и разработка компонентов, проведено комплексное тестирование программного обеспечения. Технологическая часть посвящена разработке технологии формирования SSA-представления для программ на стековых языках.

ANNOTATION

The settlement explanatory note is devoted to the development process of a stack programming language compiler. In the research part, various features of stack programming languages are considered, their comparison is carried out, conclusions are drawn about the field of applicability of stack languages. Then the typing algorithms for stack languages are described, and the structure and execution model of the WebAssembly VM are considered. The structure of the compiler has been defined in the design part, the design and development of components have been completed, and comprehensive software testing has been conducted. The technological part is devoted to the development of SSA representation generation technology for programs in stack languages.

РЕФЕРАТ

РПЗ 40 с., 3 рис., 8 табл., 5 источн., 0 прил.

СОДЕРЖАНИЕ

1 Аннотация 4
ВВЕДЕНИЕ 9
1 Анализ синтаксиса и семантики стековых языков программирования 10
1.1 Аналитический обзор стековых языков программирования 10
1.2 Сравнительный анализ стековых языков программирования 14
1.2.1 Сравнение подходов к построению синтаксиса 14
1.2.2 Сравнение подходов к типизации и оптимизации программ 16
1.2.3 Выводы 17
1.3 Анализ средств вывода типов стековых языков программирования . . . 18
1.4 Анализ устройства байткода и модели исполнения WebAssembly 22
2 Проектирование, разработка и тестирование компилятора стекового языка программирования 26
2.1 Анализ процесса компиляции 26
2.2 Анализ задания и выбор технологии, языка и среды разработки 30
2.3 Разработка структуры программного обеспечения 32
2.4 Разработка компонентов приложения 33
2.4.1 Разработка интерфейса командной строки 34
2.4.2 Разработка парсера 37
2.4.3 Разработка статического анализатора 37
2.4.4 Разработка кодогенератора 37
2.4.5 Описание стратегий, способов, методов и проведение тестирования. 37
3 Разработка технологии 38
ЗАКЛЮЧЕНИЕ 39
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 40

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Алгоритм Хиндли-Милнера – алгоритм вывода типов в полиморфных системах типов, автоматически определяющий типы выражений на основе их структуры и правил типизации без явных аннотаций (при соблюдении условий применимости системы)

Виртуальная машина – программная (либо программно-аппаратная) среда, реализующая абстрактную вычислительную модель и обеспечивающая выполнение программ в заданном формате (байткод, промежуточное представление) независимо от конкретной аппаратной архитектуры

Гомоморфизм – отображение между алгебраическими структурами, сохраняющее заданные операции (структуру), то есть переводящее результат операции в образ результата при отображении

Интерпретатор – программа, которая выполняет исходный код, по-командно анализируя и исполняя команды «на лету»

Комбинатор – функция высшего порядка, задающая способ комбинирования других функций или программных фрагментов

Компилятор – программа, выполняющая преобразование исходного текста программы на языке программирования в эквивалентную форму представления (как правило, объектный код, машинный код или промежуточное представление), пригодную для последующего исполнения или дальнейшей трансляции

Копирующий сборщик мусора – алгоритм автоматического управления памятью, при котором доступные (достижимые) объекты копируются в новую область памяти, а старая область очищается целиком

Лексема – минимальная значимая единица исходного текста программы, выделяемая на этапе лексического анализа (например, идентификатор, ключевое слово, литерал, оператор, разделитель)

Моноид – алгебраическая структура, состоящая из множества и одной ассоциативной бинарной операции, имеющей нейтральный элемент

Нетерминал – символ формальной грамматики, который может быть заменён последовательностью терминалов и/или нетерминалов по правилам вывода и используется для построения синтаксических конструкций языка

Нитевой код – способ представления программы в виде последовательности ссылок (адресов) на подпрограммы или примитивы исполнения, используемый для ускорения интерпретации и уменьшения объёма кода в стековых системах

Обратная польская запись – форма записи выражений, в которой операторы следуют за операндами (постфиксная нотация)

Рантайм – (анг. Runtime, время исполнения) имеет несколько знчений; обозначение времени исполнения программы; окружение программы, в котором она исполняется

РБНФ – расширенная форма Бекуса-Наура, нотация задания формальных грамматик, расширяющая БНФ средствами записи повторений, альтернатив, опциональных элементов и группирования

Стек – структура данных, организованная по принципу «последним пришёл – первым вышел» (LIFO), поддерживающая операции добавления и извлечения элементов с одной стороны, называемой вершиной стека

Терминал – символ формальной грамматики, который не подлежит дальнейшему разложению по правилам вывода и входит в алфавит конечной строки языка

.NET CIL – .NET Common Intermediate Language, промежуточный язык (байткод) платформы .NET, в который транслируются программы и который исполняется средой выполнения (CLR) или преобразуется в машинный код

.NET CLR – .NET Common Language Runtime, виртуальная машина платформы .NET, исполняющая программы на языке .NET CIL

Mark-sweep-compact – алгоритм сборки мусора, включающий фазу пометки достижимых объектов (mark), последующую очистку неиспользуемых объектов (sweep) и уплотнение памяти (compact) для устранения фрагментации

S-выражение – скобочный формат записи, где операция представлена скобками, внутри которых на первом месте стоит оператор, а затем перечисляются операнды

ВВЕДЕНИЕ

1 Анализ синтаксиса и семантики стековых языков программирования

1.1 Аналитический обзор стековых языков программирования

Начиная с семидесятых годов прошлого века было создано некоторое количество стековых языков. Далее рассмотрены несколько из них, каждый из которых привнес нововведения в области стековых языков. В обзор не включены, однако также заслуживают внимания:

– конкатенативный язык программирования Kitten, который продолжает идеи языка Cat в статическом анализе и представляет способ явного указания типов;

– .NET CIL и байткод Java, которые также являются языками под стековые виртуальные машины (.NET CLR и JVM соответственно).

Forth – первый известный стековый конкатенативный язык, созданный Чарльзом Муром в начале 1970-х как расширяемый язык, для которого ключевые слова можно добавлять прямо во время работы системы, и таким образом сформировать собственное подмножество языка под конкретную предметную область. Синтаксис основан на обратной польской нотации или постфиксной нотации: программа – это последовательность слов, разделённых пробелами. Интерпретатор идет слева направо по программе и ищет лексемы в словаре доступных слов и либо выполняет их, либо трактует как число и кладёт на стек. Язык и среда исполнения едины: система работает в режимах интерпретации и компиляции, переключаясь между ними. Для добавления нового слова или отложенных вычислений используется режим компиляции, для исполнения – режим интерпретации. Стандартной библиотеки в привычном смысле нет – функциональность наращивается подключаемыми наборами слов (word sets), зависящими от конкретной реализации. При этом ANS Forth (стандарт языка) задаёт базовый набор слов «core word set» со стековыми операциями, арифметикой, сравнениями, управлением потоком и примитивами памяти. Система типов у Forth «бестиповая»: данные представлены машинными словами фиксированной разрядности, а корректность трактовки содержимого стека контролирует

программист, что повышает гибкость и предсказуемость времени выполнения за счет единой семантики для любых данных, но усложняет статический анализ и проверку ошибок. Для Forth предложена оптимизация занимаемого программой пространства техникой «нитевого кода»: программа по сути является последовательностью безусловных переходов, каждый из которых ведет на инструкции, выполняющие ту или иную команду. Работа с памятью осуществляется через явные операции чтения и записи по адресу, что дает низкоуровневое управление. Область применения Forth – прошивки, встроенные и ресурсно-ограниченные системы, системы реального времени и космическая техника.

На основе идей Forth в 2001 году Манфред фон Тун представил язык Joy как попытку формализации логики стековых языков: каждая команда в Joy обозначает унарную функцию вида «stack -> stack»: на вход подаётся стек данных (состояние программы), в процессе применения команды происходит его модификация и передача следующей команде, значения и подпрограммы при этом также передаются через стек. Программу на Joy можно рассматривать как математическую функцию «stack -> stack» без побочных эффектов [1]. Язык является динамически типизированным, за проверку типов несет ответственность программист, однако ему в этом могут помогать типы, описанные явно в объявлениях функций. Синтаксис Joy также основан на постфиксной записи. Фон Тун вводит идею комбинаторов – функций высшего порядка, которые принимают другие функции (в терминах стековых языков их принято называть цитатами) и образуют некий алгоритм обработки данных (например, комбинатор линейной рекурсии «primrec» принимает цитату тела рекурсии и цитату условия рекурсии). В стандартной поставке присутствует несколько библиотек, сгруппированных по областям применения: библиотеки «agglib» и «seqlib» содержат обобщённые операции над агрегатами (неупорядоченными множествами, строками и списками) и последовательностями, «numlib» включает числовые функции и численные методы, а «mtrlib» предназначена для работы с матрицами [2].

В 2006 году Кристофер Диггинс опубликовал несколько технических отчетов, в рамках которых описывает алгоритм статической типизации для стековых языков, в качестве примера используя свой экспериментальный язык Cat. Cat наследует идею Joy: любая программа рассматривается как преобразование вида «stack -> stack», а основная операция – композиция таких преобразований, реализуемая простой конкатенацией лексем в исходном тексте программы. В плане синтаксиса Cat аналогичен предыдущим и использует обратную польскую запись. Грамматика показана на листинге 1.

Листинг 1 — РБНФ грамматики Cat

```
program    ::= { term } ;
term       ::= word | literal | quote | definition ;

definition ::= "define" name program ";" ; (* глобальное и
единственное определение *)
quote      ::= "[" program "]" ; (* цитата/замыкание *)

literal    ::= number | bool;
word       ::= name | "+" | "-" | "*" | "/" ;
name       ::= letter { letter | digit | "_" | "-" } ;
number     ::= ["+"|"-"] digit { digit } ;
bool       ::= "true" | "false" ;
digit      ::= "0"-"9" ;
letter     ::= "a"-"z" | "A"-"Z" ;
```

Статическая типизация в контексте стековых языков подразумевает, что в момент компиляции возможно построить конфигурацию стека (его размер и «форма» – типы значений, которые на нем лежат) для произвольного момента времени исполнения. Таким образом вывод типов открывает путь к статическому анализу и проведению трансформаций. Подробности статического вывода типов описываются в разделе 1.3. В языке присутствует несколько встроенных команд, их описание приведено в таблице 1. Помимо них в язык встроены арифметические, логические, команды и команды сравнения чисел. Язык использует три типа данных – числа, булевы значения и цитаты.

Таблица 1 — Встроенные команды Cat

Команда	Описание
apply	Применить цитату с вершины стека к остальному стеку.
quote	Захватить вершину стека в цитату.
compose	Выполнить композицию двух цитат и положить на стек цитату-результат композиции.
dup	Положить на стек дубль вершины стека.
pop	Удалить значение с вершины стека.
swap	Поменять значение на вершине стека с значением под ним.
cond	Комбинатор ветвления, выбирает элемент на стеке или элемент под ним в зависимости от условия.
while	Комбинатор для цикла «пока».

Еще одним ответвлением от Joy стал язык Factor, впервые опубликованный в 2003 году Славой Пестовым. Преемственность Factor прослеживает по нескольким деталям: синтаксис на основе обратной польской записи, аналогичная идея использования и способ создания новых комбинаторов. Ключевая особенность – это первый прикладной высокоуровневый стековый язык общего назначения. Так его можно назвать благодаря богатой стандартной библиотеке (такая включает в себя, например, веб-фреймворк и парсер XML) и инструментарию разработки, собранном в одной IDE и включающим в себя компилятор с развитой системой анализов, интерактивный отладчик, браузер документации, инспектор объектов. Как и Joy, язык динамический, однако существует отличие: при объявлении новой функции программист должен указать ее тип функции, в свою очередь компилятор проверяет этот тип на соответствие содержимому функции. Внутренний механизм этой проверки описывается как абстрактная интерпретация программы: компилятор (а точнее, модуль «Stack-checker» внутри него) симулирует эффекты команды на абстрактном стеке, при встрече ветвлений анализирует обе ветви и унифицирует состояния, а несовместимость размера или

формы стека превращает в ошибку времени компиляции. На основе анализа типов компилятор проводит высокоуровневые и низкоуровневые оптимизации. Также дополнительно используются динамические проверки времени исполнения. Все эти особенности помогаю языку Factor быть вполне применимым для решения задач прикладного программирования.

Наконец рассматривается спецификация для одноименных стекового языка и виртуальной машины WebAssembly (сокращенно Wasm), которые были разработаны и выпущены в 2017 году группой компаний, в том числе W3C, Mozilla, Google, Microsoft и Apple. Wasm определяет низкоуровневый переносимый байткод, архитектуру и семантику набора команд под виртуальную стековую машину, запускаемую преимущественно (но не только) в веб-браузерах наравне с другими движками. В исходных целях проектирования Wasm фиксируются требования к компактности двоичного представления, быстрой однопроходной валидации и компиляции, а также к «песочнице», пригодной для запуска недоверенного кода с производительностью низкоуровневого кода. Как стековый язык Wasm использует неявный стек данных: инструкции потребляют значения с вершины стека и помещают результаты обратно, при этом реализация не обязана хранить «настоящий» стек – спецификация описывает интерпретацию стека как набора анонимных регистров. Это первый из представленных в обзоре языков, который является целью компиляции, а не сам компилируется во что-либо. Поддержка компиляции в Wasm имеется для множества высокоуровневых языков, в первую очередь для полностью компилируемых языков C/C++/ Rust. Для языка существует два представления: текстовое (формат «Wat» с синтаксисом на S-выражениях), удобное для чтения человеком, и бинарное, понимаемое виртуальной машиной. Подробнее устройство байткода Wasm и его модели исполнения рассмотрено в разделе 1.4.

1.2 Сравнительный анализ стековых языков программирования

1.2.1 Сравнение подходов к построению синтаксиса

Таблица с сравнением языков по синтаксису представлена в таблице 2.

Таблица 2 — Сравнение синтаксиса языков (сводная таблица)

Язык	Форма записи	Особенности синтаксиса
Forth	Постфиксная	Управляющие конструкции реализуются как слова времени компиляции.
Joy	Постфиксная	Цитаты используются как данные, управление через комбинаторы («i», «ifte», «primrec/linrec/binrec»).
Cat	Постфиксная	Синтаксис близок к Joy.
Factor	Постфиксная	Обязательное указание эффектов над стеком при объявлении функций.
Wasm	S-выражения	Нетрадиционная форма записи синтаксических конструкций

В общем случае в рассмотренных языках прослеживается следующая закономерность. Для языков, которые используются непосредственно программистами для написания программ вручную, применяется характерный конкатенативный синтаксис, что соответствует изначально заложенным в языки данного множества концепциям минималистичности и простоты. Вероятно, некоторую роль в сохранении такой формы играет фактор историчности: исторически первый стековый язык (Forth) обладал конкатенативным синтаксисом.

В то же время синтаксис языков, которые являются целями трансляции, представлен более широким разнообразием. В частности Wasm использует S-выражения – тоже одна из достаточно распространных форм синтаксиса для функциональных языков. S-выражения также соответствуют концепции простоты (РБНФ синтаксиса таких языков имеет относительно малый размер).

Таким образом, отличительная черта синтаксиса стековых языков – минималистичный дизайн, который упрощает как механизмы разбора программ за счет упрощенного парсера, так и работу программиста за счет меньшего объема требуемых знаний о синтаксисе и высокой степени

читаемости (исходя из убеждения, что чем проще синтаксис, тем проще его читать).

1.2.2 Сравнение подходов к типизации и оптимизации программ

Сравнение подходов к типизации приведено в таблице 3.

Таблица 3 — Сравнение подходов к типизации

Язык	Тип типизации	Доступный анализ программ
Forth (1971)	Бестиповой	Корректность интерпретации содержимого стека обеспечивается соглашениями и выявляется на этапе исполнения.
Joy (2001)	Динамическая	Динамическая проверка типов при выполнении.
Factor (2003)	Динамическая	Модуль «stack-checker» выполняет роль статического анализатора: абстрактная интерпретация эффектов слов на абстрактном стеке и проверка согласованности ветвей.
Cat (2006)	Статическая	Вывод типов и верификация формы и размера стека, гарантии согласованности ветвлений и инвариантов циклов.
Wasm (2017)	Статическая	Однопроходная валидация модуля по типам и структурированному управлению, предотвращение underflow и несогласованности типов до исполнения.

Согласно таблице 3 в процессе развития стековые языки постепенно улучшают свои возможности динамического и статического анализа. Бестиповость сменяется динамическими типами (предложена идея описания типов), затем были предложены алгоритмы вывода, позволяющие фиксировать типы в момент компиляции.

Основные техники оптимизации показаны в таблице 4.

Таблица 4 — Специфичные техники оптимизации для стековых языков

Оптимизация	Описание
Нитевой код (Forth)	Представление программы как последовательности ссылок на слова; снижает размер кода и удешевляет интерпретацию.
Разворачивание слов времени компиляции (Forth)	Трнсляция управляющих конструкций в примитивные переходы и последовательности слов, уменьшая накладные расходы управления.
Инлайнинг цитат и макрорасширение (Factor)	Снижение накладных расходов вызовов и диспетчеризации за счёт разворачивания и специализации на этапе компиляции.

Оптимизации в стековых языках можно условно разделить на две группы: преобразования представления и диспетчеризации команд (ускоряют интерпретатор/виртуальную машину) и оптимизации, использующие результаты анализа семантики (стек-эффекты, типы, структура управления), что позволяет убирать лишние проверки и преобразовывать программу без нарушения корректности. Стековые языки изначально поддерживают техники первой группы вследствие своей структуры (например, нитевой код Forth). С развитием подходов к статическому анализу становятся доступны оптимизации из второй группы – Cat, Wasm активно применяют оптимизации, основанные на эквивалентных преобразованиях структуры программы, для увеличения скорости работы и снижении размера веса программы.

1.2.3 Выводы

В пути развития стековых языков видна тенденция: изначально работающие на низких уровнях абстракций благодаря простоте рантайма (в сравнении с нестековыми языками), высокой переносимости и оптимизациям, доступным при меньшей стоимость за счет структуры программ и модели исполнения (например, нитевой код Forth), стековые языки обзаводятся формальной базой, которая позволяет точно описывать работу программ и проводить развитой статический анализ и трансформации кода, нацеленные на уменьшение объема потребляемой памяти и увеличения скорости работы.

Одной из вершин достижений стековых языков можно считать Wasm. Исходные цели проектирования предлагают во многом подходящие для использования стекового языка условия, на основе которых выросла система, сочетающая в себе сразу несколько изначально трудносовмещаемых преимуществ: высокая портируемость под разные платформы (в сравнении с регистровыми виртуальными машинами), но при этом большая производительность (например, в сравнении с традиционными движками ECMAScript) за счет низкоуровневых интерфейсов и оптимизаций и верифицируемость (и оттого частично безопасность исполнения). Wasm вбирает в себя теоретические и практические наработки в области стековых языков, решая с помощью них сложную задачу исполнения прикладного кода с учетом заданных требований.

В результате анализа можно сделать вывод, что современная ниша стековых языков – это быстрые виртуальные машины, запускаемые на широком спектре устройств. Это утверждение также подтверждается фактом, что спецификации самых распространенных виртуальных машин (JVM, .NET CLR, Wasm) используют стековую модель исполнения кода.

1.3 Анализ средств вывода типов стековых языков программирования

Использование стека данных в качестве единого хранилища порождает несколько возможных проблем типизации. Первая – проблема несовпадение типов, когда на вход некоторой команде с вершины стека поступают данные неправильного типа. Вторая – проблема исчерпания стека, когда команда при выполнении пытается взять данные с пустого стека. Такие ошибки приводят к аварийным завершениям программ и тем самым снижают их надежность. В то же время от подобных ошибок можно защититься, если в момент компиляции программы статически определять состояния стека во все будущие моменты времени исполнения и в случае нахождения ошибки блокировать компиляцию программы как небезопасной. Просчет возможных состояний стека для программы на стековом языке называется выводом ее типа.

В стековых языках тип для некоторого выражения определяется через эффект, который оказывает это выражение на конфигурацию стека данных при выполнении. В свою очередь конфигурация стека состоит из размера стека (сколько данных на нем лежит) и «формы» стека (данные каких типов на нем лежат). Задача вывода типов для стековых языков сводится к нахождению типа для всей программы – как программа поменяет конфигурацию стека при выполнении.

В общем случае выводом типов в языках программирования называется процесс, который позволяет установить неуказанный явно тип некоторого выражения в тексте программы по ограничениям окружения, где это выражение встречается. Разные языки обладают разными возможностями вывода в зависимости от размера области программы, которая используется для вывода типа. Некоторые языки допускают простые выводы, основанные на анализе только самого выражения, для которого выводится тип. Например, вывод типов в язык программирования Java ограничивается автоматическим определением типа переменной при ее объявлении по выражению, которое инициализирует данную переменную. Другие языки идут дальше и позволяют выводить типы по совокупности выражений и совокупности переменных, задействованных в этих выражениях в рамках функций. К примеру, Rust, который позволяет выводить тип переменной из контекста выражений, где эта переменная используется – так, если функция возвращает целое 32-битное число, то для гипотетической переменной, из которой делается возврат значения и которая может быть объявлена где угодно внутри такой функции, выводится соответственно тип целого 32-битного числа (то есть тип вывелся исходя из как минимум двух выражений, где переменная была задействована). Наконец существуют языки, вывод типов в которых осуществляется в рамках всей единицы трансляции – зачастую это языки с функциональной парадигмой. В таком случае типы для всей программы можно не указывать вовсе, так как все они будут выведены неявно при компиляции.

Для стековых языков алгоритмы вывода типов всей программы (последний случай, рассмотренный в предыдущем абзаце) актуальны, так как

зачастую программисту затруднительно самому вычислить тип программы, что и порождает ошибки работы со стеком данных.

Для вывода типов в функциональных языках используется алгоритм Хиндли-Милнера. Вкратце алгоритм при запуске над несколькими выражениями сводится к двум шагам: сначала вычисляются ограничения, которые задаются формой выражений (например, одна переменная равна другой или переменная участвует в сложении, а потому должна быть складываемой), ограничения составляют систему уравнений, для которой вторым шагом ищется решение – такой набор типов, при подстановке которых, выражения останутся непротиворечивыми. Если набор найден, то типы выведены успешно, иначе – ошибка типизации.

Согласно идее, предложенной Кристофером Диггинсом, для стековых языков использование алгоритма Хиндли-Милнера также возможно, но с поправкой на row-полиморфизм. Row-полиморфизм – полиморфизм по размеру строки («row»), состоянию стека данных. Все команды row-полиморфны и оперируют всем стеком сразу, который состоит из хвоста (типовой вектор, обозначается большими латинскими буквами) и единичных значений, лежащих на вершине стека (обозначаются маленькими латинскими буквами либо названиями типов, например «a» и «Bool»). Для цитат используется запись со скобками, показывающая отложенность описанных вычислений (пример, конфигурация стека, на котором лежит цитата «A (A -> B)»).

Диггинс демонстрирует возможность статического вывода типов на примере своего экспериментального стекового языка Cat. Для встроенных команд приводятся типы, показанные на листинге 2.

Листинг 2 — Типы встроенных команд Cat

```
// Применение цитаты, которая приводит конфигурацию S к
конфигурации R, к стеку S
apply   : (S (S -> R) -> R)
// Формирование цитаты, путем захвата значения с вершины стека
quote   : (S a -> S (R -> R a))
// Композиция двух цитат
compose : (S (B -> C) (A -> B) -> S (A -> C))
```

Продолжение листинга 2

```
// Дублирование значения на вершине стека
dup     : (S a -> S a a)
// Удаление значения с вершины стека
pop     : (S a -> S)
// Перестановка двух значений с вершины стека
swap    : (S a b -> S b a)
// Примитив ветвления, выбирающий то или иное значения со
стека в зависимости от значения Bool
cond    : (S Bool a a -> S a)
// Примитив циклических операций, выполняющий тело (цитата на
вершине) пока верно условие (цитата под вершиной)
while   : ((S -> R Bool) (R -> S) S -> S)
```

Стоит обратить внимание на ограничения, которые ставят описанные в листинге 2 типы. Для условного оператора «if» тип требует, чтобы обе ветви, принимая на вход одну и ту же конфигурацию стека, возвращали тоже одну и ту же измененную конфигурацию стека (при этом конкретные значения на стеке, конечно, могут различаться). Для цикла «while» тип требует, чтобы после выполнения цикла конфигурация стека оставалась такой же, как и на момент начала цикла.

Алгоритм вывода типа заключается в последовательном «сцеплении» типов команд, образующих программу. В своем техническом отчете Диггинс приводит текстовое описание алгоритма и пример, в котором происходит вывод типа для программы «[dup] apply». В результате применения алгоритма получается следующая цепочка типов: «A a -> A a (A a -> A a a) -> A a a» (примечание, в отчете приводятся только начальная и конечная конфигурации стека, но аналогичным образом возможно построить и промежуточные конфигурации). Как видно, цитата и команда «apply» взаимно уничтожаются, из-за чего результирующий тип эквивалентен применению одиночной команды «dup» с точностью до промежуточных конфигураций.

Предложенный алгоритм имеет ограничение, заключающееся в невозможности вычислять рекурсивные типы. Из-за этого ограничения нельзя вычислить тип для программ, которые имеют рекурсивный поток управления. Также нельзя вычислить тип программ, предполагающих рекурсию за счет

использования конструкции «dup apply» (объяснить это можно так: «apply» предполагает, что на стеке лежит некоторая цитата, которая переводит стек под собой из начального состояния в некоторое другое, но при этом «dup» говорит, что сама цитата также является частью этого начального состояния под собой). Такие ограничения можно снять, если расширить систему типов дополнительными конструкциями.

Таким образом для стековых языков возможно статически вывести форму и размер стека, и в случае наличия ошибок заранее их заметить. Признаком исчерпания стека служит наличие значений на вершине стека в начальной конфигурации программы – то есть программа ожидает, что на стеке до ее запуска уже что-то лежит, хотя это не так. Признак несовпадения типов выражается невозможностью найти решение системы уравнений в процессе очередной сцепки типов команд в программе.

1.4 Анализ устройства байткода и модели исполнения WebAssembly

Байткод WebAssembly (или сокращенно Wasm) спроектирован как низкоуровневая переносимая платформа с околонативной производительностью вычислений. Поддерживается 2 формата описания программ Wasm: бинарное представление, понимаемое виртуальной машиной Wasm, и текстовое представление в синтаксисе S-выражений, удобное для чтения человеком. В зависимости от цели, одну и ту же программу возможно представить в обоих вариантах.

Единицей трансляции Wasm является модуль – описанный спецификацией Wasm файл, содержащий коды программ, наборы метаданных, информацию об импортах и экспортах и другие составляющие. Модуль является самостоятельной единицей, готовой к запуску на виртуальной машине. Для представления чисел используется формат little-endian (младшие разряды чисел записываются в младших адресах). При кодировании векторов (последовательностей значений) в начале блока данных записываются 4 байт значения длины векторов, а затем кодируемые данные. Строки кодируются как байтовые вектора utf-8.

Содержимое модуля состоит преамбулы (значение «\0asm» и 4 байт номера версии) и основной части – набор секций. Каждая секция состоит из байта-идентификатора, 4 байт размера секции и полезной нагрузки в соответствии с идентификатором секции. Секции могут идти в произвольном порядке, но рекомендуется располагать их по мере увеличения идентификаторов.

Помимо модуля Wasm определяет также нескольких других сущностей, которые описываются в модуле:

– функции – блоки исполняемого кода;

– таблицы ссылок – механизм реализации указателей на функции, динамических вызовов;

– глобальные переменные и константы – набор типизированных изменяемых или неизменяемых значений фиксированного типа;

– линейная память – непрерывные последовательности байт;

– теги – механизм исключений.

Сущности каждого вида объединяются в индексные пространства, которые используются для ссылок между секциями. Если сущность из адресного пространства задается несколькими секциями (что верно для функций, таблиц и линейной памяти, у которых объявление и определение разделены на разные секции, подробнее в таблице 5), то нумерация в этих секциях идет параллельно (например, функция с индексом 1 задается объявлением с индексом 1 и определением с индексом 1). Внутри индексного пространства нумерация сквозная и начинается с 0. Сначала нумеруются импортированные сущности в порядке перечисления в секции импорта, затем аналогично в порядке перечисления в соответствующих секциях нумеруются внутренние сущности.

Описание каждой секции приведено в таблице 5.

Таблица 5 — Описание секций модуля

Ид.	Название	Описание
0	Custom section	Игнорируется виртуальной машиной, хранит произвольные данные. Может использоваться для различных целей, в том числе для отладки.
1	Type section	Объявления доступных для использования типов функций (типы параметров и типы результатов). Остальные секции ссылаются на данную при необходимости указать тип.
2	Import section	Объявления сущностей, которые импортируются в модуль для работы. Определения импортируемых сущностей предоставляют модули, которые их экспортируют.
3	Function section	Объявления типов для определенных в модуле функций, имеет параллельную с секцией Code нумерацию.
4	Table section	Объявления таблиц ссылок, имеет параллельную с секцией Element нумерацию.
5	Memory section	Объявления линейной памяти.
6	Global section	Объявления глобальных переменных.
7	Export section	Объявления публичных сущностей, которые могут быть импортированы другими модулями.
8	Start section	Объявление индекса функции, которую необходимо вызвать при инстанцировании модуля.
9	Element section	Объявления элементных сегментов – заполнения таблиц.
10	Code section	Объявления тел функций.
11	Data section	Объявление сегментов данных.
12	Data count section	Опциональное указание числа сегментов данных (Data section), которое ускоряет валидацию модуля.
13	Tag section	Объявления тегов для реализации механизма исключений.

Вызов функций возможен либо напрямую через указание индекса нужной функции либо косвенно через таблицу ссылок. Для операций над

данными используется стек данных. Чтобы выполнить некую операцию, необходимо положить на стек требуемые данные, операция берет операнды со стека и кладет на стек результат выполнения. Функции действуют аналогично. Аналогично описанию в разделе 1.3, тип функции описывает преобразование верхушки стека, которое осуществляет функция.

Спецификация Wasm использует идею «непрозрачных» указателей, которая выражается в том, что с точки зрения программы представление указателей не раскрывается, они выступают лишь значениями, которые ссылаются на что-то. Оперирование указателями осуществляется через отдельные команды. Указатели нельзя сохранять в линейную память – только в таблицы ссылок.

Wasm различает восстановимые и невосстановимые ошибки. Невосстановимые ошибки называются «trap» и приводят к аварийному завершению программы. Восстановимые ошибки реализованы как исключения. При возникновении исключения указывается его тег и машина раскручивает стек вызовов программы, пока не найдется обработчик соответствующего тега или стек вызовов будет раскручен полностью, и программа аварийно завершится.

Таким образом, спецификация WebAssembly предлагает низкоуровневую виртуальную машину. За счет низкого уровня и небольшого количества абстракций и ряда инженерных решений (таких как стековая архитектура, непрозрачные указатели) достигается во-первых малая удельная стоимость исполнения команд, что делает программы Wasm сопоставимы по скорости с нативными программами, а во-вторых снижается сложность реализаций виртуальной машины для различных аппаратных платформ (чем проще машина, тем проще ее реализовать) и вследствие этого увеличивается география устройств, для которых реализована виртуальная машина и возможно исполнение Wasm-программ. Эти преимущества делают Wasm подходящей платформой для запуска аппаратно-независимых программ на множества устройств, что активно используется в веб-разработке и облачных сервисах.

2 Проектирование, разработка и тестирование компилятора стекового языка программирования

2.1 Анализ процесса компиляции

Компилятор служит промежуточным звеном между человеком, пишущим программу, и компьютером, который будет исполнять заданные программой команды. Таким образом, человек и компилятор образуют пару, ответственную за предоставление описания задач для компьютера в понятном для него виде – в низкоуровневых командах. Чтобы программу назвали хорошей, она должна мочь выполнять поставленные задачи, притом делать это с минимальным количеством ошибок и эффективно по количеству используемых ресурсов (далее по тексту под оптимальностью программы понимается ее эффективность использования ресурсов компьютера, а под оптимизацией – увеличение степени оптимальности). Каждый из пары «человек и компилятор» имеет свою зону ответственности в достижении заданных для программы требований. Компилятор отвечает частично за минимизацию ошибок и частично за эффективность, проверяя и совершенствуя то, что описал человек на исходном языке.

Таким образом, можно выделить 3 задачи компилятора:

1) трансляция программ на исходном языке в целевое представление;

2) проверка программы на наличие ошибок (верификация);

3) оптимизация программы.

В зависимости от специфики предметной области эти задачи могут изменяться как в количестве, так и в определениях. Список выше представлен для предметной области стековых языков.

Процесс компиляции заключается в последовательном преобразовании входных данных – текста программы на исходном языке (в контексте этой работы, подмножестве языка Cat) – в некоторое количество промежуточных форм и наконец в финальную форму выходного файла.

Каждая из форм программы служит некоторой цели. Исходная форма обычно приспособлена для работы человека – она должна обеспечивать удобство читаемости программы, ее создания и редактирования. Стоит

отметить, что описанная в этой форме программа может уступать в эффективности, но должна превосходить в понятности и читаемости остальные формы программы.

Финальная форма полностью соответствует формату файла, ожидаемому на выходе в результате компиляции, то есть модулю WebAssembly. За счет исходной и финальной форм компилятор выполняет первую задачу.

Возможные назначения промежуточных форм более разнообразны и зачастую связаны с выполнением какой-либо из задач компилятора (которые описывались выше).

Как уже было сказано в разделе 1.3, для проверки программы на стековом языке на наличие ошибок работы со стеком возможно использовать один из методов статического анализа – типизацию программы. Результат применения этого метода – представление типа программы. Это и есть первое из промежуточных представлений. Его назначение – выполнение второй поставленной выше задачи компилятора.

Для выполнения третьей задачи компилятора, оптимизации, было создано немало методов и техник. Оптимизация основывается на переписывании программы таким образом, чтобы эффекты со стороны сохранились в нетронутом виде, но сама программа стала более оптимальной. На текущий момент в индустрии компиляторов популярностью пользуется форма SSA-представления (single static assignment, единственное статическое присвоение). SSA-форма удобна для проведения большого количества оптимизаций, так как за счет своих свойств заметно упрощает алгоритмы анализа и трансформации. Подробнее об SSA-представлении рассказывается в 3. Таким образом, еще одно промежуточное представление – SSA закрывает последнюю стоящую перед компилятором задачу.

Для представления процесса трансляции можно воспользоваться методологией IDEF0. Контекстная диаграмма показана на рисунке 1.

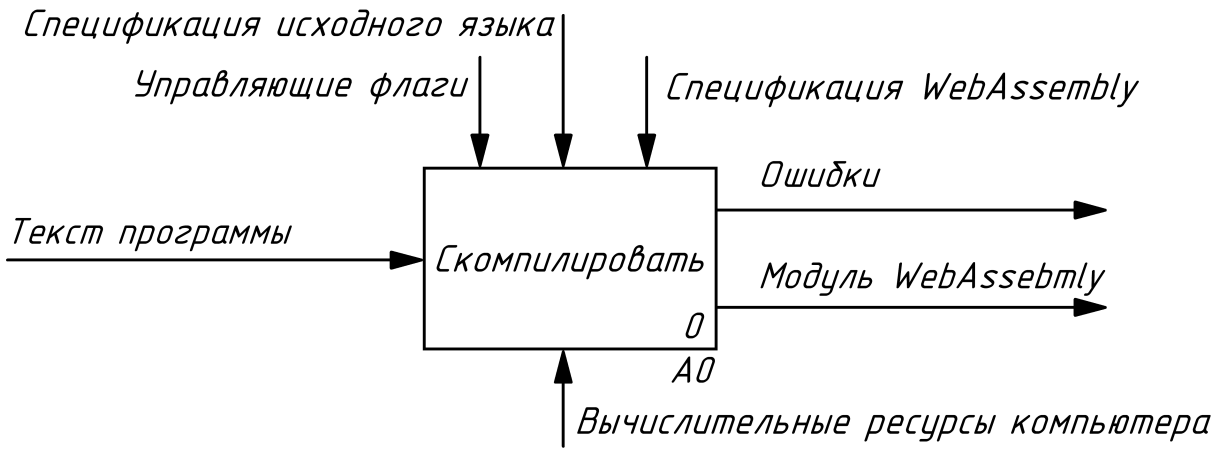

Рисунок 1 — Контекстная диаграмма

Контекстная диаграмма отражает компилятор как единую функцию A0 «скомпилировать». Входом функции является текст программы, поступающий слева. Управляющими воздействиями выступают спецификация исходного языка, управляющие флаги и спецификация WebAssembly, задающие правила разбора исходной программы, режим работы компилятора и требования к результирующему модулю. Выходами функции являются модуль WebAssembly и сообщения об ошибках. Механизмом выполнения служат вычислительные ресурсы компьютера, на которых реализуется процесс компиляции.

Диаграмма декомпозиции на рисунке 2 уточняет процесс компиляции.

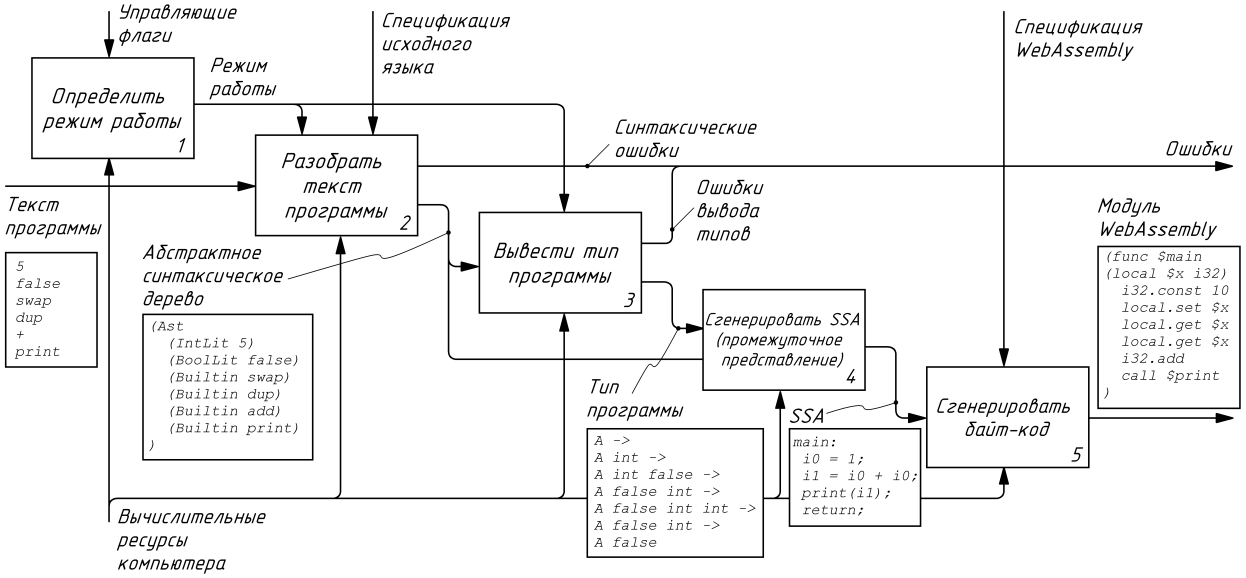

Рисунок 2 — Диаграмма декомпозиции

Процесс компиляции разделен на пять функций. Согласно техническому заданию, компилятор должен реализовывать несколько режимов работы:

проверка программы на корректность и генерация модуля WebAssembly или только проверка. Этой логикой является блок 1, который по управляющим флагам определяет режим работы компилятора. Результатом работы блока является выбранный режим работы, который далее управляет прохождением программы по последующим этапам обработки. В блоке 2 выполняется разбор текста программы в соответствии со спецификацией исходного языка. Результатом этого этапа является абстрактное синтаксическое дерево (AST). При невозможности корректного разбора формируются синтаксические ошибки, которые выводятся как один из результатов работы компилятора.

В блоке 3 по построенному абстрактному синтаксическому дереву выводится тип программы. Если вывод типов завершается успешно, получается представление типа программы, иначе формируются ошибки вывода типов. Полученное представление типа используется на следующем этапе. В блоке 4 на основе абстрактного синтаксического дерева и выведенного типа строится SSA-представление, выступающее промежуточной формой для дальнейшего преобразования программы. Наконец, в блоке 5 по SSA-представлению и с учетом спецификации WebAssembly генерируется итоговый байт-код в виде модуля WebAssembly.

Связи между блоками отражают последовательное преобразование форм программы: исходный текст преобразуется в абстрактное синтаксическое дерево, затем для него выводится тип, после чего строится SSA-представление и на его основе создается конечный модуль WebAssembly. Вычислительные ресурсы компьютера выступают механизмом выполнения для этапов разбора, вывода типов, построения SSA-представления и генерации байт-кода, то есть обеспечивают фактическую реализацию всех преобразований программы. На рисунке 2 также показаны листинги, демонстрирующие различные представления обрабатываемой программы в зависимости от этапа обработки.

По диаграмме на рисунке 2 можно сделать вывод: компиляция программы возможна только при ее корректности. Для некорректной программы не получится построить абстрактное синтаксическое дерево, либо же вывести ее тип. Обе структуры данных необходимы для построения SSA-

представления, которая в свою очередь используется для генерации байт-кода. Тогда для некорректной программы не существует представления в виде байт-кода, а следовательно компиляция невозможна.

2.2 Анализ задания и выбор технологии, языка и среды разработки

В соответствии с требованиями технического задания необходимо разработать программу, которая может выполнять трансляцию кода на исходном языке. Компилятор должен обеспечивать поддержку ряда синтаксических конструкций, представляющих исходный язык. Модули байт-кода, являющиеся результатом работы компилятора, должны соответствовать спецификации WebAssembly [3]. Программное обеспечение должно работать под управлением ОС Linux и иметь интерфейс командной строки.

Исторически к программам-компиляторам предъявляются требования по скорости работы, нативности, наличию интерфейса командной строки. Иными словами, привычный компилятор – скомпилированное нативное CLI-приложение без сборщика мусора.

Вышеперечисленные требования сужают диапазон подходящих языков программирования до нескольких вариантов: C, C++, Rust, Zig. В результате по совокупности факторов был выбран язык Rust. Компилятор данного языка обеспечивает автоматический контроль за состоянием памяти без использования сборщика мусора, сам язык обладает наиболее строгой системой типов (среди предложенных) [4]. Указанные особенности Rust позволяют писать безопасное решение и недопускать ошибки в программном обеспечении. В таблице 6 показаны результаты сравнения языков программирования.

Для разработки на данном языке принято использовать программу Visual Studio Code, поэтому она выбрана в качестве среды разработки.

Поскольку процесс компиляции можно описывать как последовательность вызовов функций, поэтапно преобразующих код от исходного языка до байт-кода, рационально использовать структурный подход. Структурный подход также является идиоматичным при разработке на Rust.

Компилятор Rust автоматически контролирует состояние памяти, проверяя корректность работы с памятью на этапе компиляции. Проверка осуществляется за счет концепций «владения» и «заимствования». При написании программы программист должен объяснить компилятору, как используется память в программе, а компилятор должен проверить, что такое использование безопасно. Преимуществом такого подхода по сравнению с другими автоматическими решениями является то, что все проверки осуществляются до начала исполнения программы.

Таблица 6 — Сравнение свойств языков программирования

	C	C++	Zig	Rust
Работа с памятью	Ручная	Ручная	Ручная	Автоматическая
Компиляция в нативный код	Да	Да	Да	Да
Зрелость и стабильность	Да	Да	Нет	Скорее да
Современные методы разработки	Нет	Да	Да	Да

Под зрелостью и стабильностью подразумевается наличие стабильного канала выпуска новых версий программного обеспечения. Стабильным каналом принято называть канал, в рамках которого сохраняется обратная совместимость между версиями программного обеспечения, а также осуществляется долгосрочная поддержка версий.

Наконец критерий «современные методы разработки» учитывает поддержку языком принятых в профессиональном сообществе стандартов написания кода и программных продуктов.

При создании программного обеспечения целесообразно проводить разработку нисходящим способом, как одним из рекомендуемых [5]. Версионирование программного обеспечения осуществляется при помощи инструмента git.

2.3 Разработка структуры программного обеспечения

Согласно подразделу 2.2 в рамках разработки был выбран структурный подход. Для работы при таком подходе необходимо уточнить структурную схему программного решения.

В проведенном в подразделе 2.1 анализе процесса компиляции выделено несколько функций: определение режима работы, разбор текста программы, вывод ее типа, генерация SSA и генерация байт-кода. Уместно использовать метод пошаговой детализации и для каждой функции выделить свой структурный компонент, который будет отвечать за выполнение этой функции. Также возможно дополнить список компонентами, которые не показаны на диаграмме декомпозиции A0 из-за недостаточной степени детализации, но имеют заметную с точки зрения структуры роль. Общий список компонентов и их описания приведены в таблице 7.

Таблица 7 — Компоненты программного обеспечения

Название	Назначение компонента
Программа-компилятор	Точка входа в программное обеспечение, управление остальными компонентами, определение режима работы
Компонент интерфейса командой строки	Предоставление интерфейса взаимодействия с программных обеспечением по средствам терминала
Парсер	Разбор текстов программ, построение абстрактного синтаксического дерева
Статический анализатор	Проведение анализов и трансформаций программы: вывод типов, построение SSA-представления, оптимизация SSA-представления

Продолжение таблицы 7

Название	Назначение компонента
Кодогенератор	Создание модулей WebAssembly согласно SSA-представлению

На рисунке 3 изображена структурная схема проекта, составленная по управлению.

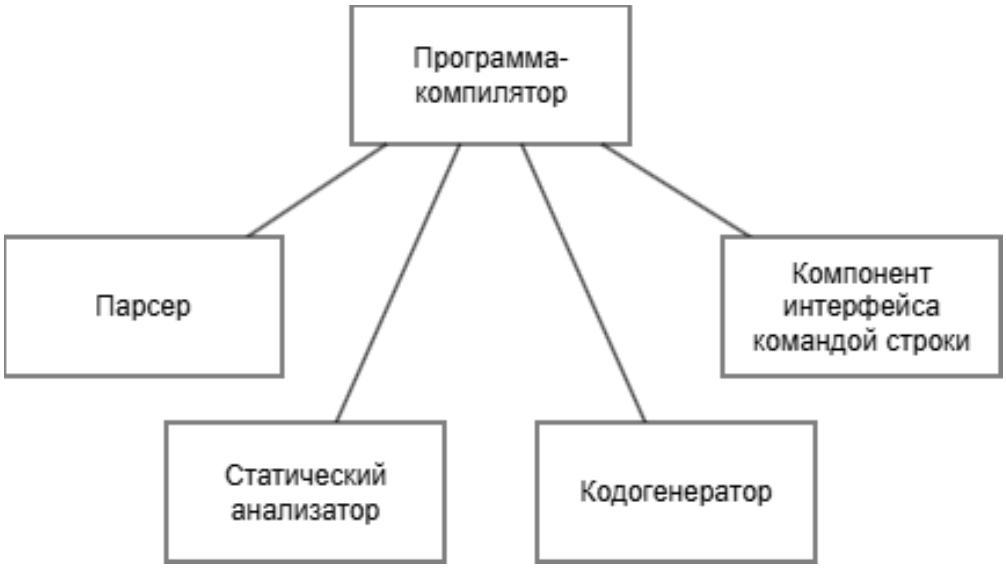

Рисунок 3 — Структурная схема программного обеспечения

На приведенной схеме компонент «Программа-компилятор» является главной частью программного обеспечения, который обеспечивает управление остальными модулями. Компонент интерфейса командной строки предоставляет программисту способ общения с программой при помощи терминала и занимается разбором вводимых программистом запросов, которые затем передает управляющему компоненту. Парсер, статический анализатор и кодогенератор совместно выполняют функции трансляции программы согласно процессу на рисунке 2.

2.4 Разработка компонентов приложения

После определения структуры приложения необходимо разработать его компоненты согласно выбранному нисходящему подходу. Для повышения наблюдаемости и качества отладки, каждый компонент должен уметь генерировать для оператора сообщения разной степени критичности, уровень критичности должен задаваться операторов при вызове программного

обеспечения. Уровни критичности сообщений приведены далее в порядке убывания приоритета:

– сообщения с невосстановимыми ошибками,

– сообщения с предупреждениями,

– отладочные сообщения.

Также компоненты должны обладать низким уровнем сцепленности и высокой связностью, что положительно сказывается на технологичности программы.

Далее рассмотрена разработка каждого компонента согласно подразделу 2.3.

2.4.1 Разработка интерфейса командной строки

Интерфейс командной строки представляет собой способ взаимодействия с программным обеспечением при помощи терминала. Поскольку согласно техническому заданию необходимо обеспечить работоспособность программы в рамках Unix-подобной операционной системы Linux, дальнейшее описание подразумевает использование стандартного для Unix-подобных систем терминала на основе командной оболочки Unix. Взаимодействие строится на основе модели «запрос-ответ»: оператор вызывает требуемую программу при помощи написания команды, которая состоит из названия программы, набора передаваемых параметров, а также операторов управления стандартными потоками ввода, вывода и ошибок.

В командной оболочке Unix принято соглашение об именовании параметров (флагов): перед именем параметра идет префикс в виде одного или двух символов «-», для именования параметра могут использоваться длинные и короткие имена. Короткие имена отличаются от длинных тем, что состоят только из одного символа. Для коротких имен префикс состоит из одного символа «-», для длинных – из двух «-». У параметров могут быть аргументы, которые в случае наличия записываются следом за именем параметра через пробельные символы. Аргументы могут быть также у программы как таковой, а не у конкретного параметра, в таком случае аргументы перечисляются в конце команды вызова.

При завершении программа возвращает код ответа, обозначающий результат вызова. Если код ответа равен 0, то программа завершилась успешно. В ином случае считается, что в работе программы имелись ошибки. Интерфейс компилятора должен соответствовать данной схеме обратной связи.

Для построения грамматики необходимо определить требования к интерфейсу. Согласно техническому заданию, необходимо поддержать указание различных режимов работы, что может осуществляться через выбор одного из двух доступных параметров: компиляция программы или только верификация. В качестве аргумента программы необходимо поддержать указание пути до файла с текстом программы, а также флаг, указывающий на имя результирующего файла с байт-кодом. Должны быть флаги, регулирующие уровень сообщений от компонентов компилятора. Далее приводится таблица 8, содержащая информацию о параметрах.

Таблица 8 — Допустимые параметры при вызове компилятора

Короткое имя	Длинное имя	Аргументы	Описание	Обязатель-ность
«-f»	«--verificate»	–	Включение режима верификации	Нет
«-c»	«--compile»	–	Включение режима компиляции	Нет
«-q»	«--quite»	–	Отключение вывода всех сообщений	Нет
–	–	Имя файла	Название для входного файла с программой	Да
«-o»	«--output»	Имя файла	Название для генерируемого выходного файла	Нет
–	«--log»	«debug» или «warn» или «error»	Определение общего уровня сообщений	Нет

Продолжение таблицы 8

Короткое имя	Длинное имя	Аргументы	Описание	Обязатель-ность
–	«--log-parser»	«debug» или «warn» или «error»	Определение уровня сообщений парсера	Нет
–	«--log-analyzer»	«debug» или «warn» или «error»	Определение уровня сообщений статического анализатора	Нет
–	«--log-codegen»	«debug» или «warn» или «error»	Определение уровня сообщений кодогенератора	Нет

Имя выходного файла по умолчанию соответствует имени входного файла без постфикса расширения. Например, для файла «foo.kol» выходной файл будет обладать именем «foo».

Значения для параметров уровня сообщений имеют следующую логику:

– при значении «debug» должны отображаться отладочные сообщения и остальные более критичные сообщения;

– при значении «warn» должны отображаться сообщения с предупреждениям и сообщения с ошибками;

– при значении «error» должны отображаться только сообщения с ошибками.

По умолчанию все параметры уровня сообщений имеют значение «warn».

Наконец возможно представить грамматику интерфейса командной строки в виде расширенной формы Бекуса-Наура (РБНФ), что и показано в листинге 3.

Правило «path» в грамматике соответствует строке с путем в рамках файловой системы Linux.

Для реализации интерфейса командной строки в экосистеме языке Rust принято использовать библиотеку Clap. Эта работа не стала исключением.

Листинг 3 — РБНФ грамматики интерфейса командной строки компилятора

```
compiler  ::= "kolenka" { param } path ;
param     ::= short | long ;
short     ::= "-f" | "-c" | "-q" | "-o", path ;
long      ::= "--verificate" | "--compile" | "--quite" | "--
output", path | "--log", log_level | "--log-parser", log_level
| "--log-analyzer", log_level | "--log-codegen", log_level |
log_level ::= "debug" | "warn" | "error" ;
```

2.4.2 Разработка парсера

2.4.3 Разработка статического анализатора

2.4.4 Разработка кодогенератора

2.4.5 Описание стратегий, способов, методов и проведение тестирования

3 Разработка технологии …

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Mathematical foundations of Joy [Электронный ресурс]. URL: https://hypercubed.github.io/joy/html/j02maf.html (дата обращения: 10.10.2025).

2. The prototype implementation of Joy [Электронный ресурс]. URL: https://hypercubed.github.io/joy/html/j09imp.html (дата обращения: 10.10.2025).

3. WebAssembly Specification 2.0 [Электронный ресурс]. URL: https://webassembly.github.io/spec/versions/core/WebAssembly-2.0.pdf (дата обращения: 11.11.2025).

4. Документация Rust [Электронный ресурс]. URL: https://www.rust-lang.org/learn (дата обращения: 29.01.2026).

5. Иванова Г.С. Технология программирования: Учебник для вузов. 3-е изд., стер. – М.: КНОРУС, 2018. 334 с.